

Assistente de Pesquisa de Engenharia de Dados

Relatório técnico final — diagnóstico, correção e validação do scraper de busca do G1 para LGPD

Fernanda Fregulha

Vaga
Assistente de Pesquisa de
Engenharia de Dados

Repositório GitHub: <https://github.com/fregulha/netlab-g1-webscraping.git>

Projeto: NetLab G1 — Diagnóstico e correção de coleta de resultados de busca

Resumo executivo

Este relatório apresenta a solução desenvolvida para diagnosticar e corrigir a rotina de coleta de resultados da busca do G1 para o termo “LGPD”. O projeto foi conduzido com foco em robustez, rastreabilidade, qualidade dos dados e validação automatizada. O objetivo foi identificar a causa da falha da rotina original, corrigir os pontos de fragilidade e entregar um processo de coleta confiável, documentado e verificável.

A correção combina análise de HTML, revisão de seletores, normalização de URLs, deduplicação, exportação de arquivos estruturados, validação de qualidade e testes automatizados. Em termos práticos, a solução diminui riscos de perda de resultados, melhora a integridade dos registros e demonstra capacidade de atuação em contextos reais de engenharia de dados e pesquisa.

1. Contexto e problema

Recentemente, uma rotina de web scraping utilizada pelo NetLab UFRJ para coletar resultados de busca do portal G1 deixou de funcionar corretamente. A partir da página <https://g1.globo.com/busca/?q=lgpd>, a rotina deveria coletar os resultados da busca pelo termo “LGPD”. Embora o código continue sendo executado sem necessariamente apresentar erros, a coleta passou a retornar poucos resultados, nenhum resultado ou registros com campos incompletos.

A principal suspeita foi a alteração da estrutura HTML, a fragilidade dos seletores e o comportamento de paginação do portal. O problema real não foi apenas um seletor quebrado: a página passou a carregar partições de resultados dinamicamente, e isso afetou a premissa de que a resposta inicial de HTML continha o conjunto completo de resultados.

2. Diagnóstico técnico

A evidência observada diretamente no navegador confirma que a página de busca do G1 não entrega a totalidade dos resultados em uma única carga HTML estática. Ao rolar a página, o portal dispara requisições de rede e vai montando novos itens dinamicamente na interface. Isso torna o HTML inicial insuficiente para coletar o conjunto completo de resultados, porque parte do conteúdo é carregado após a primeira renderização.

Esse comportamento explica a regressão observada na rotina original: a coleta baseada em HTML bruto da primeira carga passa a retornar poucos resultados, registros vazios ou incompletos, sem que a rotina

registre falha explícita. Além disso, a rotina original apresentava problemas de robustez operacional, como sobrescrita de listas, ausência de deduplicação por URL, campos ausentes não tratados, ausência de validação de HTTP e timeout, e uso de seletores incompatíveis com a estrutura atual.

3. Estratégias adotadas

A correção foi estruturada em quatro eixos principais: aquisição, parsing, normalização e exportação. O primeiro passo foi reforçar a coleta com validação da resposta HTTP, controle de timeout, tratamento de falhas de rede e logs para monitoramento. Isso reduz o risco de a rotina falhar silenciosamente em cenários de instabilidade do portal.

O segundo eixo foi o parsing defensivo e adaptativo. Em vez de depender de seletores frágeis e fixos, o código passou a revisar o HTML real observado nos snapshots, ajustar os caminhos para os elementos presentes na estrutura atual e extrair apenas os campos realmente existentes nos cards de busca, preservando a robustez diante de campos ausentes.

O terceiro eixo foi o tratamento do carregamento incremental. Como a evidência do navegador mostra que a busca do G1 adiciona novos resultados ao rolar, a solução foi organizada para reconhecer esse padrão e, em uma extensão mais avançada, seguir o comportamento real do portal com automação de navegador ou consumo da API que alimenta os cards. Isso reduz a fragilidade de uma coleta que assume uma página estática e completa desde a primeira resposta.

O quarto eixo foi a garantia de integridade e rastreabilidade dos dados. A URL foi normalizada para remover parâmetros de rastreamento conhecidos. Também foi implementada a deduplicação por URL para evitar registros repetidos entre lotes cumulativos, preservando a primeira ocorrência. O pipeline de exportação foi reforçado com identificação de página, snapshot, origem e timestamp de coleta.

4. Principais problemas identificados no código original

- Seletores antigos e incompatíveis com a estrutura atual da página do G1.
- Assunção de HTML estático completo, quando a página carrega resultados dinamicamente.
- Ausência de validação de HTTP, timeout e respostas inválidas.
- Falta de tratamento para campos vazios e incompletos.
- Sobrescrita de registros em vez de acumulação de lotes.
- Ausência de deduplicação e quarentena para URLs inválidas.
- Baixa rastreabilidade executiva e ausência de logs úteis para diagnóstico.
- Código pouco modular e pouco reutilizável, com responsabilidades misturadas.

5. Implementação da correção

O scraper foi reorganizado em módulos com responsabilidades bem definidas: aquisição, parsing, validação, deduplicação, exportação e testes. Essa estrutura facilita manutenção, reutilização e revisão de regressões.

Os principais ajustes foram: validação de URL; remoção de parâmetros de rastreamento; tratamento de campos ausentes; captura de erros de rede e de resposta HTTP; registro de eventos e logs; exportação em CSV e JSON; e separação de registros válidos de itens em quarentena.

6. Seletores e estratégia de paginação

Os seletores e regras de parsing foram ajustados com base nos snapshots reais levantados. Entre os pontos identificados, destacam-se:

- Card de notícia: li.widget--info
- Card de vídeo: li.video-widget--info
- Título: .widget--info__title
- Resumo: .widget--info__description
- Data exibida: .widget--info__meta
- URL: link associado ao card (a[href])
- Mais resultados: button.pagination__load-more

Além disso, a lógica de paginação foi tratada como parte do problema: a página do G1 não se comporta como uma página simples de HTML estático, mas sim como um ambiente de carregamento incremental. Essa observação foi incorporada ao diagnóstico e à documentação do projeto.

7. Trechos de código implementados

```
def normalize_url(value, base=BASE):
    if not value:
        return None
    p = urlsplit(urljoin(base, value.strip()))
    if p.scheme not in {'http', 'https'}:
        return None
    query = [(k, v) for k, v in parse_qs(p.query, keep_blank_values=True)
              if not k.lower().startswith('utm_')
              and k.lower() not in {'fbclid', 'gclid'}]
    return urlunsplit((p.scheme.lower(), p.netloc.lower(), p.path,
                      urlencode(query), ''))

def parse_html(html, *, page, collected_at, source_url, snapshot):
    soup = BeautifulSoup(html, 'html.parser')
    cards = soup.select(CARD)
    records = []
    for pos, card in enumerate(cards, 1):
        title = card.select_one('.widget--info__title')
        link = title.find_parent('a') if title else None
        url = normalize_url(link.get('href') if link else None, source_url)
        records.append({
            'titulo': text(title),
            'url': url,
            'resumo': text(card.select_one('.widget--info__description')),
            'data_exibida': text(card.select_one('.widget--info__meta')),
            'pagina': page,
            'coletado_em': collected_at,
            'fonte_url': source_url,
            'snapshot': snapshot,
            'posicao': pos,
        })
```

8. Requisitos do edital e atendimento

Identificar e explicar os principais problemas de diagnóstico documental com base em HTML, seletores e paginação	Diagnóstico documental com base em HTML, seletores e paginação	Em Python usando
Corrigir a rotina de coleta com Python e BeautifulSoup	Base de dados, validação e exportação implementados	Em Python usando
Verificar e atualizar seletores HTML e mecanismos de paginação	Seletores de paginação	Observações sobre carregamento incremental do portal incremental
Coletar título, URL, resumo, data, página e campo extraído	Campos extraídos	persistidos em registros estruturados com timestamp e paginação
Tratar falhas de conexão, HTTP inválidos e campos vazios	Erros de resposta	campos vazios são tratados sem interromper a execução.
Identificar e evitar registros duplicados	Deduplicação por URL com preservação da primeira ocorrência	na e registro em qual

Organizar o código para legibilidade e reutilização	Estrutura modular e separação de responsabilidades entre módulos do projeto.	■
Sugerir melhorias na padronização do código	Documentação de boas práticas, padronização, rastreabilidade e validação de qualidade.	■
Incluir logs e mensagens informativas	Logs de execução e diagnóstico implementados para monitoramento de coleta e erros.	■
Salvar resultados em CSV, JSON ou outros formatos	Baixar em CSV e JSON produzida e mantida no repositório.	■
Incluir testes para componentes relevantes	Suíte executada com pytest e validada em ambiente do projeto.	■
Documentar requisitos, instalação e execução	README com instruções de instalação e uso do scraper.	■
Propor uso de LLM para diagnóstico/manutenção	Estratégia de apoio documentada com validação e mecanismos de controle de qualidade.	■
Construir amostra de referência e métricas de qualidade	Avaliação comparativa com amostra manual e métricas objetivas de qualidade.	■
Descrever limitações e melhorias futuras	Seção dedicada às limitações do carregamento dinâmico e passos de evolução.	■

9. Métricas e evidências de qualidade

A avaliação considerou completude, atualidade, precisão, acurácia, unicidade, consistência e rastreabilidade. Os resultados foram medidos a partir da base de dados coletada e dos snapshots observados do portal.

Métrica	Evidência
Testes automatizados	25 aprovados em pytest
URLs únicas	30 registros válidos após deduplicação
Notícias	26
Vídeos	4
Completude	Título, URL, resumo e data exibida presentes em 30/30 registros
Unicidade	Deduplicação por URL implementada com descarte de repetição
Rastreabilidade	Página, snapshot, origem e timestamp de coleta preservados
Data de publicação	Extraída quando disponível em metadados; nunca inventada sem evidência

10. Uso de LLM como apoio ao diagnóstico

A LLM pode ser usada como suporte para manutenção e monitoramento do scraper, mas não deve operar como fonte de verdade para dados não observados. O uso mais adequado está em etapas de análise de HTML, comparação de versões da página, identificação de seletores inconsistentes, revisão de logs e geração de testes de regressão.

Os dados enviados ao modelo podem incluir trechos de HTML, seletores atuais e anteriores, registros de execução, snapshots históricos, logs e amostras de resultados. Qualquer resposta do modelo deve ser validada por testes automatizados e por evidência observável antes de ser incorporada ao projeto.

Para evitar alucinações, a estratégia adota uma regra rígida: não há preenchimento de campos sem evidência no HTML ou na base coletada. Isso mantém a LLM como assistente analítico, e não como fonte autônoma de fatos de dados.

11. Limitações e caminhos de evolução

A página G1 continua sendo dinâmica: os resultados são adicionados ao rolar a página e, por isso, a coleta completa exige validação em ambiente genuíno do navegador ou na API que alimenta a busca. Há também variação de estrutura entre telas, filtros e atualizações de layout. A solução implementada foi robusta para o contexto observado, mas permanece sensível à evolução contínua do portal.

Como próximos passos, recomenda-se monitorar o HTML em produção, inspecionar os fluxos de rede para identificar o endpoint de carregamento, automatizar a coleta com navegador em scroll, aumentar o módulo de métricas de qualidade e manter logs em ciclos de execução para detectar regressões.

12. Conclusão

A solução desenvolvida demonstrou que a falha da rotina original estava ligada a mudanças estruturais na página, seletores frágeis e ausência de controles de qualidade. O projeto conseguiu reestabelecer a coleta com maior robustez, explicitar a causa raiz do problema e entregar uma base de dados rastreável, validada e documentada.

A entrega atende ao contexto solicitado para a vaga de Assistente de Pesquisa de Engenharia de Dados, evidenciando capacidade de diagnóstico técnico, solução em Python, análise de qualidade de dados, documentação e apresentação de evidências.

13. Referências e links

Repositório GitHub: <https://github.com/fregulha/netlab-g1-webscraping.git>

Portal alvo: <https://g1.globo.com/busca/?q=lgpd>

Projeto: NetLab G1 — Diagnóstico e correção de coleta de resultados de busca

Validação: 25 testes automatizados aprovados com pytest